

Activity H1 - Solve the "Hidden Problem" Behind the Stated Problem

Title: From Surface Problem to Real Problem

Goal

Train yourself to see the *real* problem, not just the stated one.

Instructions

You are given a "client statement":

"Our exam system is too slow. We want a Python script to speed it up."

1. Identify at least 3 possible *real* problems this statement might hide.

- Hidden problem 1: Poor database performance causing slow queries
- Hidden problem 2: Inefficient backend logic/code
- Hidden problem 3: Server overload due to high user traffic

2. For each hidden problem, state:

- What data would you need to confirm it?
 - Problem 1: Database query logs, response time matrices
 - Problem 2: Code profiling results
 - Problem 3: Server load stats, number of concurrent users

3. Choose ONE hidden problem and re-frame it as a clear technical problem statement (2-3 lines):

New problem statement:

The exam system is slow due to inefficient database queries that increase response time under load.

4. Design a very small Python experiment that could validate or reject your reframed problem (pseudo-code is fine):

python

Experiment idea (what will you measure, on what data, how?)

measure query execution time

import time

start = time.time()

run_sample_queries() # stimulate database queries

end = time.time()

print("Execution time:", end - start)

Question 12 - Multiple Choice Question

Topic: Design & Implementing Python Code

Goal

Question: Explain an in-memory data structure strategy.

Instructions

Prerequisites

"You receive a stream of log lines (text). You must detect 'suspicious' activity in real time."

1. Give three different solution ideas, each using a distinct main technique or data structure.

- Approach A (e.g., rule-based / regex / thresholds):

Rule-based using regex to detect suspicious keywords

- Approach B (e.g., statistical / sliding window / counters):

Sliding window with counters to track unusual frequency

- Approach C (e.g., simple ML / anomaly detection concept):

Machine learning anomaly detection

2. For each approach, list pros and cons (at least one each).

- Approach A - Pros: simple fast Cons: limited accuracy
- Approach B - Pros: Detect patterns over time Cons: Needs tuning
- Approach C - Pros: high accuracy Cons: Complex needs training

3. Choose one approach to implement first and justify:

I choose approach because

I choose approach B because it balance efficiency & detection accuracy without heavy computation

4. Sketch high-level Python structure (modules/classes, not full code):

python

Modules / classes / key functions list

```
# modules / classes
```

```
class log Reader:
```

```
    pass
```

```
class Detector:
```

```
    def analyze (self, logs):
```

```
        pass
```

```
class Alert System
```

```
    def send - alert (self):
```

```
        pass
```

Project 10 - Task Management Application (Elegant Pipeline)

The goal is to design a (simple) application, (constrained)

Goal

Build a tool that is not only simple, but also elegant and beautiful

Function

Implement a simple "pipeline" for processing a list of tasks

- Each task has an ID, description, and status ("todo", "doing", "done")
- You must:
 - Load tasks from a JSON file
 - Filter by status
 - Sort by ID
 - Print a clean summary

Constraints (you impose on yourself)

1. Maximum 25 lines of real Python (excluding blanks and comments).
2. No function longer than 7 lines.
3. Names must be meaningful and short.

Steps

1. Design your data model in 2-3 lines:

Task = {id: int, desc: str, status: str} list of task stored in JSON

2. Decide at least one thing you will not do to keep it minimal:

No GUL or complex validation to keep code minimal

3. Write your code here (respecting constraints):

python

Elegant pipeline solution

4. Reflection:

- What did you have to sacrifice (if anything) to keep elegance?

I sacrificed detailed error handling and extra features for simplicity

Python:

```
import json
def load - task (file):
    return json.load (open (file))
def filter - task (tasks, status):
    return [t for t in tasks if t["status"] == status]
def sort - task (tasks):
    return sorted (tasks, key = lambda x: x["id"])
def show (tasks):
    for t in tasks:
        print (f'{{t["id"]}}: {{t["desc"]}}
        [{{t["status"]}}])
```


Activity H4 – Complexity and Scaling Thought Experiment

Title: How Will Your Solution Break at Scale?

Goal

Develop realistic intuition about performance and scaling.

Instructions

Take ANY non-trivial function or algorithm you have recently written (search, summarization, scoring, etc.).

1. Describe what it does, in one line:

Search for a value in a list

2. Estimate its time complexity in Big-O notation, and justify in 1–2 lines:

- Big-O: $O(n)$
- Because: It checks each element once

3. Scaling scenarios – imagine input sizes:

- $n = 10^3$ (1,000)
- $n = 10^6$ (1,000,000)
- $n = 10^9$ (1,000,000,000)

For each, predict: will it still be usable (Y/N) and why?

n	Usable (Y/N)	Reason (1 short sentence)
10^3	Yes	Small dataset, fast
10^6	Yes	Still manageable
10^9	No	Too slow for linear scan

4. Suggest one change to improve scalability and briefly describe how it helps:

Use binary search (O(log n)) to reduce time complexity.

Activity H5 - Adversarial Thinking: Break Your Own Design

Title: Attack Your Own Program

Goal

Develop robustness and defensive mindset.

Instructions

Choose one of your own projects or mini-tools (file reader, API client, CLI, etc.)

1. Write a short description of what it is supposed to do:

A file reader that loads and display file content

2. Now wear the hat of an *attacker* or malicious user.

List 5 ways to misuse / break it:

- Attack 1: Upload very large file
- Attack 2: Invalid file format
- Attack 3: malicious file content
- Attack 4: Empty file
- Attack 5: Unauthorised file access

3. For each attack, suggest one defence in plain language:

- Defence for attack 1: Limit file size
- Defence for attack 2: Validate file type
- Defence for attack 3: Sanitize input
- Defence for attack 4: Handle empty cases
- Defence for attack 5: Add authentication

4. Implement at least one of these defences in your code and paste the updated snippet:

python

Hardened code snippet:

```
def read-file (file):  
    if file.size > 1e6:  
        return "file too large"  
    if not file.name.endswith(".txt"):  
        return "Invalid format"  
    return file.read()
```

Activity 115 - Cross-Domain Transfer

Title: Apply Known Pattern to a New Domain

Goal

Practice lifting a known idea/pattern from one domain to a different one.

Instructions

Pick a pattern you already know well, e.g.:

- Sliding window
- Two-pointer technique
- Dynamic programming table
- Producer-consumer
- Map-reduce style
- Pipeline of filters

1. Name of pattern: Sliding window

2. Original domain where you learned it (e.g., arrays, strings):

Arrays

3. New domain to apply it in (e.g., logs, time series, learning activities, medical data):

log monitoring

4. Describe how you would adapt the pattern to this new domain, in 5-8 lines (can include pseudo-code):

Use a time based window to track logs within last few seconds. Continuously update window as new logs arrive. Remove old logs outside window. count suspicious event in window. trigger alert if threshold exceeded.

5. Optional: implement a small prototype showing this transfer:

python

Pattern transfer prototype

```
window = []  
def process(log):  
    window.append(log)  
    if len(window) > 5:  
        window.pop(0)  
    if "error" in " ".join(window):  
        print("Alert!")
```


Activity 117 - Meta-Cognition: Your Personal Problem-Solving Manual

Title: Write Your Own "How I Solve Problems" Guide

Goal

Turn your tacit skills into an explicit, reusable system.

Instructions

1. List the first three things you usually do when you get a new problem (be honest):

- Step 1: Read problem
- Step 2: Think of solution
- Step 3: Start coding

2. Now, design your ideal 6-8 step process (what a "future you" would always do):

1. Understand problem clearly
2. Break into smaller parts
3. Identify constraints
4. Explore multiple solutions
5. Choose best approach
6. Implement step-by-step
7. (optional) Test thoroughly
8. (optional) Optimize

3. For each step, write one question you will ask yourself:

- Step 1 question: What exactly is required?
- Step 2 question: Can simplify it?
- Step 3 question: What are constraints?
- Step 4 question: Are there better approaches?
- Step 5 question: Is this optimal?
- Step 6 question: Does it work correctly?

4. Final instruction to yourself (one sentence):

"When I feel stuck, I will break the problem into smaller parts
and try again."